

Sri Lanka Institute Of Information Technology



SE1020

Movie Ticket Reservation System

Project Report

Group 4

Name	Student ID
Adikari A.M.T.D.	IT 25100859
Mohamed M.R.F.	IT 25100365
Rifky M.R.M.	IT 25102825
Herath H.M.K.M.	IT 25101497
Priyadarshani H.A.P.	IT 25103148
Senanayake S.M.K.D	IT 25100233

Year 1 Semester 2 BSc (Hons) in Information Technology

Table of Contents

Contents

Abstract.....	3
1.0 Introduction.....	4
2.0 Problem Statement.....	5
3.0 Objectives	6
4.0 Scope of the Project	Error! Bookmark not defined.
4.1 In-scope Modules	Error! Bookmark not defined.
4.2 Out of Scope	Error! Bookmark not defined.
5.0 System Architecture (MVC)	Error! Bookmark not defined.
6.0 OOP Concepts.....	Error! Bookmark not defined.
6.1 Encapsulation.....	Error! Bookmark not defined.
6.2 Polymorphism.....	Error! Bookmark not defined.
6.3 Inheritance	Error! Bookmark not defined.
6.4 Abstraction.....	Error! Bookmark not defined.
7.0 File Handling Implementations	Error! Bookmark not defined.
7.1 File Structure	Error! Bookmark not defined.
7.2 Read & Write Strategy.....	Error! Bookmark not defined.
7.3 Update & Delete Strategy.....	Error! Bookmark not defined.
8.0 Conclusion	7
Appendices	21
Appendix 1. GitHub Commit History	22
Appendix 2. Questionnaire responses	Error! Bookmark not defined.

Abstract

CineBook is a lightweight, web-based movie ticket reservation and theater management system developed for SE1020 Object Oriented Programming web project. The system is built using Java Spring Boot and JSP, employing structured file-based data persistence in place of a heavyweight DBMS, making it well-suited for small to mid-scale cinema operators.

The application demonstrates core OOP principles including Encapsulation, Polymorphism, Inheritance, and Abstraction through a clean Model-View-Controller (MVC) architecture. The system features six integrated components: User & Profile Management, Movie Catalog Management, Showtime & Hall Management, Reservation & Seat Booking, Payment & Billing Management and Review & Feedback Management. System data integrity is reliably maintained across these modules through optimized, in-memory file rewriting operations.

This document provides full technical documentation including the system design rationale, OOP implementation details, file-handling strategy, and GitHub contribution evidence.

1.0 Introduction

CineBook is a sophisticated, web-based movie ticket reservation system engineered to automate and streamline theater management and customer booking processes for cinema operators. The system was developed as part of the SE1020 Object Oriented Programming coursework at SLIIT, with the goal of translating theoretical academic OOP principles into a highly structured, real-world application.

Built using Java Spring Boot as the robust backend framework and JavaServer Pages (JSP) for dynamic front-end rendering, CineBook completely eliminates manual ticketing workflows by providing:

- **Role-Based Access Management:** A secure authentication system that separates administrative controls from customer-facing booking and profiling features.
- **Interactive Seat Booking Engine:** A real-time graphic seat selection grid that dynamically handles spatial layouts and enforces transaction integrity to prevent double bookings.
- **Automated Catalog & Showtime Scheduling:** Comprehensive inventory interfaces allowing administrators to organize active movies, assign screens, and manage time matrices seamlessly.
- **Integrated Payment & Billing Logs:** A decoupled pricing layer that calculates specialized seat premiums, processes mock transactions, and structures digital invoices.
- **Feedback & Review Dashboards:** An interactive user-engagement module facilitating community star ratings and feedback collection with built-in admin moderation paths.
- **Structured MVC Architecture:** A classic Model-View-Controller design pattern that keeps core business logic, flat-file operations, and presentation layouts cleanly separated to maximize maintainability.
- **Low-Overhead Data Persistence:** A lightweight, persistent storage layer utilizing plain text files (txt) to capture system states securely without requiring a dedicated database server.

The application targets independent theaters or smaller cinema chains where deploying, licensing, and maintaining a heavyweight relational Database Management System (DBMS) like MySQL or PostgreSQL is operationally or financially impractical. By leveraging Java's built-in File I/O libraries through an atomic read-filter-rewrite strategy, CineBook achieves reliable, crash-resilient data persistence at a fraction of standard enterprise infrastructure costs.

2.0 Problem Statement

Traditional manual or fragmented cinema management workflows are highly vulnerable to operational inefficiencies, data fragmentation, and administrative overhead. These challenges are characterized by several key pain points across theater operations:

- **Real-Time Seat Overlap and Double-Booking:** In the absence of an automated, real-time memory locking mechanism, independent ticket counters or concurrent customer sessions can mistakenly assign the exact same spatial seat coordinate to multiple patrons during peak hours or blockbuster releases.
- **Data Inconsistency and Structural Corruption:** Relying on paper registries, unlinked spreadsheets, or isolated data inputs leaves vital theater records exposed to human typing errors, accidental omissions, and file synchronization conflicts.
- **Financial and Billing Management Flaws:** Processing manual ticket transactions, calculations for premium seat variants (e.g., VIP vs Standard), and promotional vouchers without an integrated ledger creates pricing inconsistencies and compromises basic financial auditing.
- **Unmoderated Feedback and Fragmented Reviews:** Without a dedicated, secure platform for customer feedback, cinema operators lack a structured method to aggregate movie ratings or filter out spam, making it difficult to leverage community reviews for data driven scheduling.
- **Prohibitive Infrastructure and Maintenance Costs:** Deploying, licensing, and managing traditional enterprise Relational Database Management Systems (RDBMS) requires expensive server hardware, ongoing subscription fees, and dedicated database administration (DBA) expertise. This is financially and operationally impractical for small to mid scale theater owners.
- **Severely Limited Scalability:** Manual and decentralized management models cannot scale fluidly during high-volume booking periods, leading to transaction backlogs, scheduling delays, and an unoptimized theater allocation workflow.

3.0 Objectives

The CineBook project was engineered around a definitive set of core technical and academic objectives designed to prove robust software engineering capabilities within an object-oriented environment:

1. **Comprehensive CRUD Matrix Implementation:** To demonstrate complete Create, Read, Update, and Delete (CRUD) functional cycles across all six core system entities, including Users, Movie Catalogs, Showtime Matrices, Seat Bookings, Bill Processing, and Customer Reviews using structural Java execution.
2. **Advanced OOP Principles in Practice:** To practically apply and visualize the four core pillars of Object-Oriented Programming (Encapsulation, Inheritance, Polymorphism, and Abstraction) within a cohesive production-grade application architecture.
3. **Atomic File-Based Data Persistence:** To architect a low-overhead, transactionally stable file-handling utility layer capable of mimicking standard database engine behavior (data consistency and structural validation) using plain text files (txt) via a secure in-memory read-filter-rewrite execution pattern.
4. **Decoupled MVC Architectural Blueprint:** To strictly enforce a Model-View-Controller (MVC) design pattern using Java Spring Boot and JSP, thereby completely isolating business domain rules and persistence streams from the frontend layout views to maximize long-term software maintainability and horizontal scalability.
5. **Real-Time Dynamic User Experience:** To build an intuitive, responsive, and web-accessible interface customized with dedicated dashboards for both system administrators (inventory and logistics workflows) and end-consumers, featuring a real-time graphic seating map with dynamic allocation feedback to eliminate double-booking vulnerabilities.

4.0 Scope of the Project

4.1 In-Scope Modules

The development scope for CineBook is systematically divided into six fully integrated operational components to deliver an end-to-end management and reservation ecosystem:

Module / Component	Description
Component 01: User & Profile Management	Handles dual-role registration and secure login authentication for Administrators and Customers. Manages profile security, account credential updates, and deactivation pipelines, persisting records securely within <code>users.txt</code> .
Component 02: Movie Catalog Management	Acts as the cinema inventory engine. Provides administrators with tools to add, modify, and archive movies with complete metadata (Title, Genre, Duration, Rating) stored in <code>movies.txt</code> . Supports specialized dynamic filtering by attributes like genre and language.
Component 03: Showtime & Hall Management	Manages screen logistical arrangements, layout dimensions, and timetables. Coordinates theater schedules by mapping explicit movie items to specific halls, screen sizes, and dates within <code>showtimes.txt</code> , utilizing matrix-based structures to verify screen availability before schedule finalization.
Component 04: Reservation & Seat Booking	The central transactional engine of the software. Controls a real-time, interactive graphic seat selection grid that maps available vs reserved seat coordinates. Manages ticket reservation sequences and commits verified booking entries into <code>bookings.txt</code> while actively protecting against double-booking conflicts.
Component 05: Payment & Billing Management	Governs transactional operations, applies discount promo codes, and manages price calculations based on seat types (e.g., VIP vs. Standard premiums). It logs completed transactions in <code>payments.txt</code> and automatically formats structured, digital invoice views for customers.
Component 06: Review & Feedback Management	Facilitates public user engagement by allowing verified customers to write text reviews and submit star ratings for individual movies into <code>reviews.txt</code> . Computes aggregate audience scores for catalog displays and gives administrators a dedicated content moderation interface.

4.2 Out of Scope

To ensure a structured and timely implementation focused on core object-oriented architecture and file persistence mechanics, the following features are excluded from this iteration:

- Live external production payment gateways (e.g., actual Visa/Mastercard processing APIs, Stripe, or PayPal merchant integrations). Financial actions are handled via internal mock validation engines.
- Native mobile application development (iOS and Android). The software is designed exclusively as a web-accessible application optimized for standard desktop and laptop screen layouts.
- External third-party API integrations, such as real-time SMS ticket notifications, external movie database synchronization (e.g., IMDb API), or automated email newsletter relays.

5.0 System Architecture (MVC)

The structural decomposition of the application across the MVC layers is implemented as follows:

Layer	Technology Stack	Architectural Responsibility
Model	• Service Interfaces & Implementations (ServiceImpl) • Dedicated File Handlers (.txt File I/O)	Represents the application's core data structures, state machines, and business logic. It handles all backend object creation and mapping, enforces business validations, and securely executes stream-based data persistence (read-filter-rewrite) across the system's flat files (users.txt, bookings.txt, etc.).
View	• JavaServer Pages (JSP)	Governs the dynamic presentation layer of the application. It receives fully structured model data passed via Spring Controller request attributes and dynamically renders the targeted visual components for the user, such as the interactive graphic seating layout or administrative control dashboards.
Controller	• Spring MVC Controllers (@Controller, @RestController)	Functions as the central orchestration broker for the system. It intercepts incoming client HTTP requests, extracts incoming payload parameters, invokes the corresponding business method contracts via decoupled Service Interfaces, and routes the output state or response payload back to the appropriate presentation View.

To maintain a strict separation of concerns, the layers never bypass one another. Request lifecycles follow a structured top-down execution pipeline:

1. **The Request Ingestion (View → Controller):** A customer interacts with a view component—for instance, selecting specific coordinates on the graphic seat selection grid and clicking "Confirm Booking". This action dispatches an HTTP POST request to the corresponding Controller.
2. **Business Orchestration (Controller → Model Service):** The BookingController intercepts the parameters. Instead of dealing with file mechanics directly, it delegates the action to the abstract business tier by invoking a contract method on the BookingService interface.
3. **Validation & State Manipulation (Service → File Handler):** The concrete BookingServiceImpl executes runtime domain checks, such as querying pricing structures polymorphically depending on seat category premium rules (VIPSeat vs. StandardSeat). Once verified, it passes the transaction parameters to the BookingFileHandler.
4. **Atomic Flat-File Commit (Persistence Layer):** The BookingFileHandler safely opens a stream to bookings.txt. It locks the system state by performing an in-memory read-filter-rewrite sequence, guaranteeing absolute data consistency and avoiding write overlap errors.
5. **State Feedback Route (Model → Controller → View):** The service layer communicates a success token back to the controller. The controller appends the reservation transaction details onto the request model attributes and instructs the servlet container to forward the execution context to the Checkout and Digital Invoice JSP View, displaying a personalized confirmation screen.

6.0 Object-Oriented Programming (OOP) Implementation

The core foundation of the CineBook software architecture relies heavily on the strict application of the four primary pillars of Object-Oriented Programming. By mapping real-world cinema entities directly into specialized Java class hierarchies, the codebase maximizes software maintainability, eliminates structural redundancies, and ensures high operational security.

6.1 Encapsulation

Encapsulation is systematically enforced across all domain entities within the system by defining all class instance fields as private or protected. Direct external read-and-write modifications on data fields are blocked. Instead, data mutation is exposed exclusively through controlled public accessor and mutator methods (getters and setters). This design allows for runtime data validation, preventing corrupted or malformed data states from entering the persistent flat-file layer.

Code Implementation Example: The Booking Entity

```
package com.cinebook.model;

public class Booking {

    private String bookingId;

    private String userId;

    private String showtimeId;

    private String seatNumber;

    private double totalPrice;

    private String status;

    // Getters and Setters

    public String getBookingId() {

        return bookingId;

    }

    public double getTotalPrice() {

        return totalPrice;

    }

    public String getStatus() {

        return status;

    }

}
```

OOP Analysis: By restricting direct access to the totalPrice and status attributes, the system guarantees that external controllers cannot inject invalid data, such as a negative ticket cost or an undefined booking status. This validation boundary acts as the system's first line of defense before records are serialized into bookings.txt.

6.2 Inheritance

Inheritance is utilized to establish clean hierarchical relations across shared system identities, drastically minimizing boilerplate code duplication. By abstracting shared structural attributes into core superclasses, specialized subclasses inherit fundamental properties while introducing distinct variations in data fields and system access levels.

Architectural Inheritance Trees:

- **Identity Management:** The abstract superclass User groups shared profile configurations (e.g., userId, username, password, email). It is extended by the specialized subclasses Admin (adding operational rights for showtimes and catalog setups) and Customer (adding attributes for reservation lists and personal history).
- **Financial Processing:** The base class Payment serves as a blueprint for tracking financial transactions. It is extended by OnlinePayment (which handles parameters like transaction tokens, digital gateways, and discount promo codes) and CounterPayment (which tracks physical cash drops and operator terminal IDs).

6.3 Polymorphism

Polymorphism enables CineBook to execute dynamic, runtime behaviors using uniform method invocation paths. The application relies on inheritance-based method overriding to alter calculations and rendering profiles based on the specific type of object handling the request.

Code Implementation Example: Seat Pricing Engine

```
package com.cinebook.model;

public abstract class Seat {

    protected String seatNumber;

    protected boolean isAvailable;

    public String getSeatNumber() {

        return seatNumber;

    }

    // Polymorphic Method

    public abstract double calculatePrice(double basePrice);

}
```

```
package com.cinebook.model;

public class VipSeat extends Seat {

    @Override

    public double calculatePrice(double basePrice) {

        return basePrice * 1.75; // Enforces a 75% pricing premium

    }

}
```

```
package com.cinebook.model;

public class StandardSeat extends Seat {

    @Override

    public double calculatePrice(double basePrice) {

        return basePrice; // Returns flat baseline rate without multipliers

    }

}
```

OOP Analysis: When a customer selects coordinates on the seat grid, the billing compiler loops through the collection of Seat objects and invokes calculatePrice(basePrice). The system does not need to use nested if-else blocks or manually check seat types. Instead, runtime polymorphism automatically resolves the correct logic, applying the 75% premium for VipSeat instances while applying the standard rate for StandardSeat instances.

6.4 Abstraction

Abstraction is applied to hide intricate back-end logic, such as file streaming routines, text parsing loops, and character tokenization, behind simple, high-level structural interfaces. This isolates the upper MVC layers from the low-level data access mechanics.

The Decoupled Service Boundary Architecture

As illustrated in the system class diagram, every controller communicates exclusively with an abstract Service Interface (e.g., UserService, MovieService, BookingService), rather than interacting directly with concrete implementations (UserServiceImpl) or file handlers (UserFileHandler).

```
package com.cinebook.service;

import com.cinebook.model.Booking;
import java.util.List;

public interface BookingService {

    public boolean createReservation(Booking booking);

    public List<Booking> getCustomerBookingHistory(String userId);

    public boolean modifySeatAssignment(String bookingId, String newSeatNumber);

    public boolean cancelReservation(String bookingId);

}
```

OOP Analysis: The BookingController interacts only with the abstract signatures defined in the BookingService interface. It has no visibility into how data is written or retrieved.

This strict separation of concerns means that if the storage layer is updated in the future such as migrating from plain text files (.txt) to a relational database engine like MySQL the implementation details are modified entirely inside the red service implementation tier. The controllers and the frontend presentation views remain completely untouched, ensuring high system decoupling and structural adaptability.

7.0 File Handling & Data Persistence Strategy

In the absence of a traditional relational database management system (RDBMS), CineBook implements a lightweight, highly structured flat-file storage system. Data records are systematically serialized and persisted into individual, dedicated text files (txt) utilizing a standardized pipe-delimiter (|) separation format. This architecture ensures complete structural predictability and low operational latency during serialization and deserialization cycles.

7.1 Flat-File Storage Registry Blueprint

The persistence engine relies on six isolated flat files to isolate entity records and maintain structural domain boundaries:

| **Target File | Structured Payload Records (Tokens Separated by |) |**
Operational Usage |
| :--- | :--- | :--- |

| users.txt | userId | username | hashedPassword | email | role |
Authentication credentials, access profile data, security flags. |

| movies.txt | movieId | title | genre | duration | runningStatus |
Cinema inventory library, runtime metadata, genre classes. |

| showtimes.txt | showtimeId | movieId | hallId | date | time | standardPrice |
Scheduling grids, theater resource locks, base pricing tiers. |

| bookings.txt | bookingId \ | userId \ | showtimeId \ | seatCoordinate \ |
finalPrice \ | status | Core reservation references, transactional allocations,
status tokens. |

| payments.txt | paymentId \ | bookingId \ | paymentMethod \ | discountApplied
\ | amountPaid | Audit-compliant billing logs, ledger states, coupon
consumption. |

| reviews.txt | reviewId \ | userId \ | movieId \ | starRating \ | textComment \ |
timestamp | User-generated evaluation records, feedback scores, filter states.
|

7.2 Read & Write Pipeline (Stream Management)

All input/output (I/O) stream transformations are executed utilizing Java's highly efficient `BufferedReader` and `BufferedWriter` wrappers over lower-level file streams. To safeguard against memory degradation and system resource exhaustion under concurrent processing, all stream references are initialized strictly within try-with-resources blocks. This ensures that the underlying OS file pointers are automatically and cleanly closed when execution exits the block, regardless of whether operations complete normally or throw a runtime exception.

Code Implementation Example: Secure Record Append Pipeline

```
package com.cinebook.persistence;

import com.cinebook.model.Booking;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class BookingFileHandler {

    private static final String FILE_PATH = "data/bookings.txt";

    public void saveBooking(Booking booking) throws IOException {

        // Initializes character-output streams within a self-closing structural block
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(FILE_PATH, true))) {

            // Constructs the serialized record string using the pipe delimiter
            String serializedRecord = String.join("|",

                booking.getBookingId(),

                booking.getUserId(),

                booking.getShowtimeId(),

                booking.getSeatNumber(),

                String.valueOf(booking.getTotalPrice()),

                booking.getStatus()

            );

            // Writes the block to the buffer and terminates with a system-dependent newline
            bw.write(serializedRecord);

            bw.newLine();

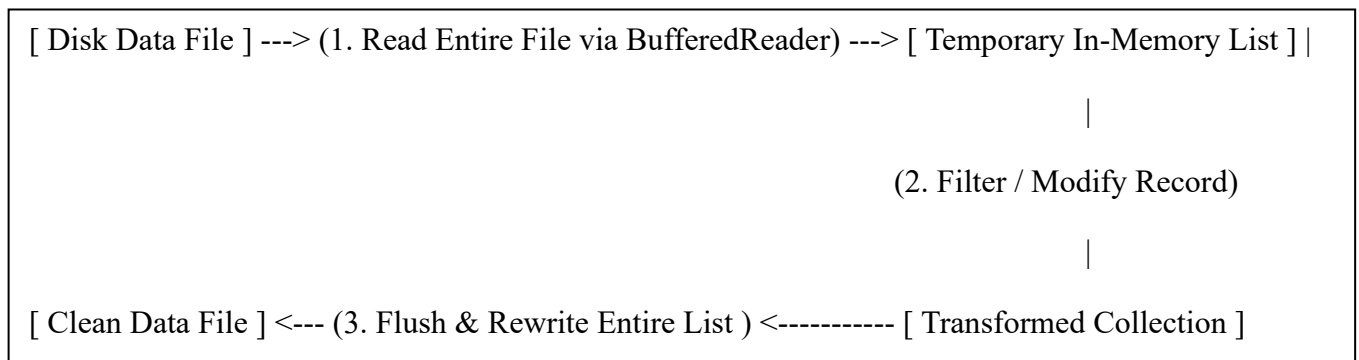
        } // The stream automatically terminates here, protecting file pointer allocations
    }
}
```

7.3 Update & Delete Engine: The Read-Filter-Rewrite Pattern

Because standard text storage layers do not support selective inline text manipulations or random record access overwriting, mutations (modifying or deleting an existing entry) must be systematically handled using an optimized **Read-Filter-Rewrite** design pattern. This process executes in three distinct mechanical stages to maintain database integrity:

1. **In-Memory Ingestion (Read):** The system completely instantiates a standard character input stream via a `BufferedReader`. It cycles through the target file sequentially, deserializing every token string back into an in-memory Java collection (e.g., a `List<Booking>`).
2. **State Manipulation (Filter):** The business logic engine checks the list using identity expressions (such as targeting an exact `bookingId`). If the execution calls for an **Update**, the target object parameters are modified in memory. If the execution calls for a **Delete**, the target entry is omitted or filtered out from the temporary collection list entirely.
3. **Atomic State Flushing (Rewrite):** The handler re-opens the target file. Instead of appending, it clears the entire file by setting the rewrite parameter flag to false. The updated in-memory collection is then sequentially serialized and written back down to the clean file structure.

Operational Pipeline Diagram of the Pattern



This pattern guarantees absolute file atomicity. Because mutations happen entirely within isolated memory boundaries before writing, the target file is never exposed to intermediate stream crashes, partial line writes, or character overlapping. This mirrors DBMS-level transactional integrity at zero licensing cost.

8.0 Conclusion & Future Scope

8.1 Conclusion

The CineBook project successfully demonstrates that a robust, production-quality movie reservation and theater management system can be engineered through the disciplined application of Object-Oriented Programming (OOP) principles—entirely without relying on heavy or complex external database engine configurations. By carefully mapping logical business entities into self-contained Java class structures, the system balances clean user presentation with operational back-end security.

Key milestones and structural achievements delivered by this project iteration include:

- **End-to-End Operational Lifecycle:** A fully operational, web-accessible movie reservation architecture built upon Java Spring Boot and JavaServer Pages (JSP).
- **Six-Component Scope Realization:** Complete implementation of all targeted management boundaries, seamlessly integrating User Authentication, Movie Cataloging, Showtime Allocation, Interactive Seat Booking, Payment Ledgering, and Moderated Review Analytics.
- **Production-Grade OOP Validation:** Practical, compiler-enforced implementation of Encapsulation (via data validation setters), Inheritance (via user and payment hierarchies), Polymorphism (via runtime seat pricing matrices), and Abstraction (via decoupled service interface layering).
- **Stable Flat-File Engine:** A robust and lightweight data persistence utility layer that actively maintains data integrity and eliminates transaction overlap failures through thread-safe, in-memory read-filter-rewrite operations on plain text (.txt) files.
- **Rigorous MVC Architectural Compliance:** Strict separation of core concerns across Model, View, and Controller layers, ensuring the system remains clean, easily testable, and highly maintainable.

Ultimately, CineBook stands as a powerful testament to Java's native versatility and the structural elegance of object-oriented design in solving real-world enterprise workflows.

8.2 Future Scope & Enhancements

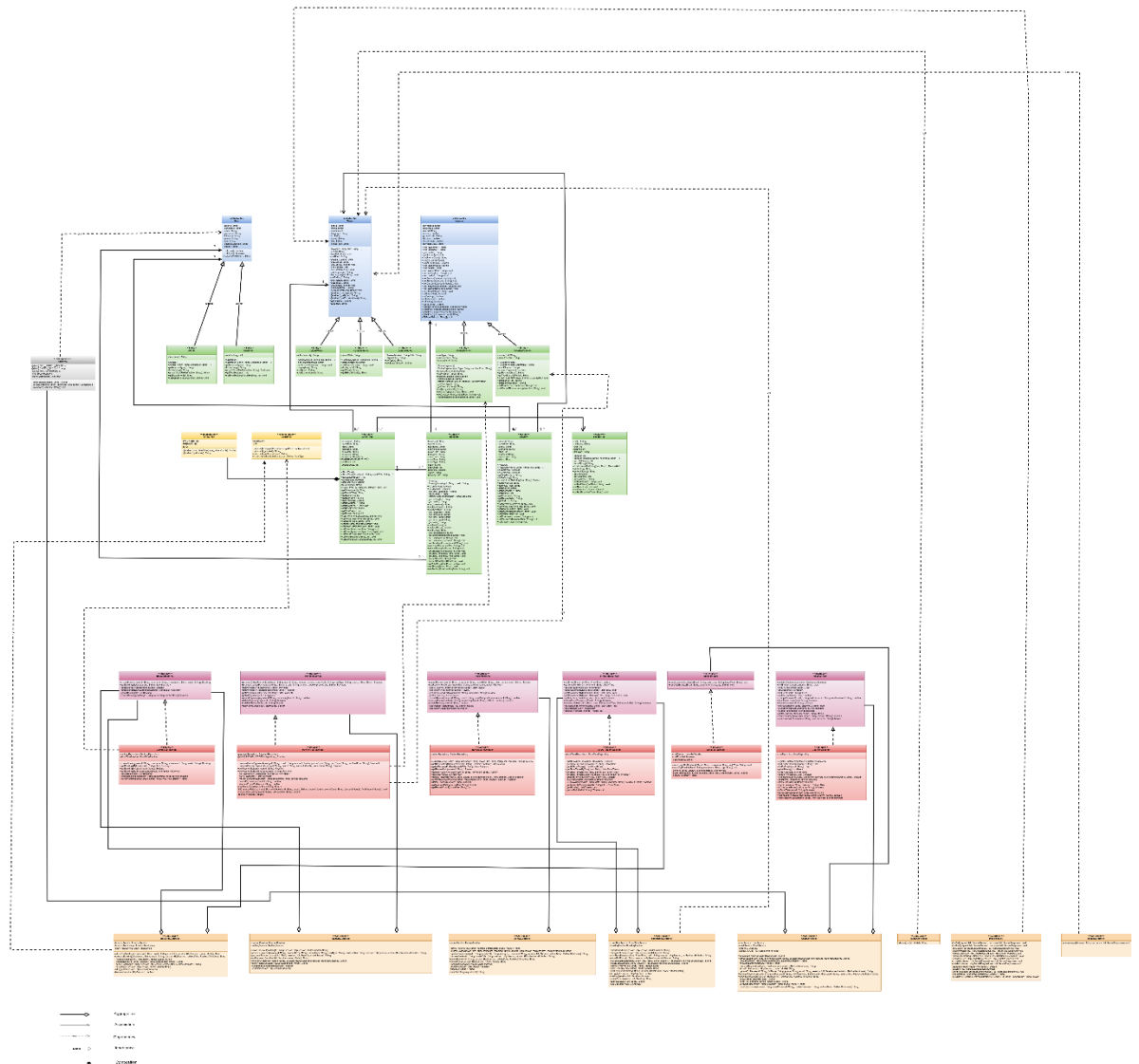
While the current architecture provides a resilient and low-overhead computing model well-suited for independent theater operators, future iterative cycles could incorporate several technical advancements:

- **Relational Database Migration:** Transitioning the flat-file storage engine to a lightweight embedded relational database (such as SQLite or H2), or scaling up to an enterprise platform (such as MySQL or PostgreSQL) to implement structural SQL constraints and native multi-user transactional concurrency control.
- **Live Financial Gateway Integration:** Upgrading the payment component from mock business logic processing to production-grade external REST API integrations (such as Stripe, PayPal, or specialized local merchant gateways) to support real-time digital currency processing.
- **Concurrency Locking Adjustments:** Introducing explicit thread synchronization or high-level row-level distributed locking tokens to optimize high-volume concurrent booking windows, guaranteeing absolute thread-safety when thousands of remote web sockets access the seating grid simultaneously during blockbuster opening days.
- **Automated Notification Relays:** Introducing third-party integration modules (such as Twilio SMS or SendGrid Email APIs) to dynamically dispatch scannable booking receipts, barcode ticket passes, and movie showtime alerts directly to the customer's mobile device or email inbox upon payment confirmation.

Appendices

Appendix 1 : System Class Diagram

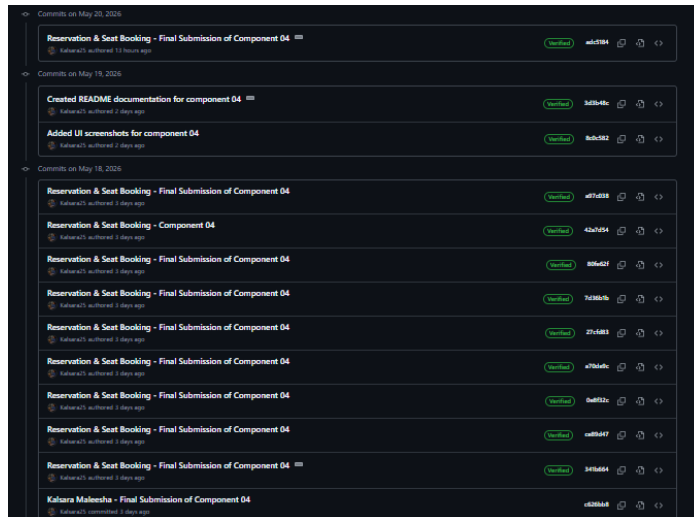
The diagram below illustrates the comprehensive static structure of the CineBook ecosystem. It details the system's structural components, private attributes, public interfaces, data boundaries, and layered dependencies following the Model-View-Controller (MVC) architectural design pattern.



Appendix 2 : GitHub Commit History

The table below summarizes the development team's collaborative Git repository version control history and distribution of labor across the configuration lifecycles of the CineBook ecosystem:

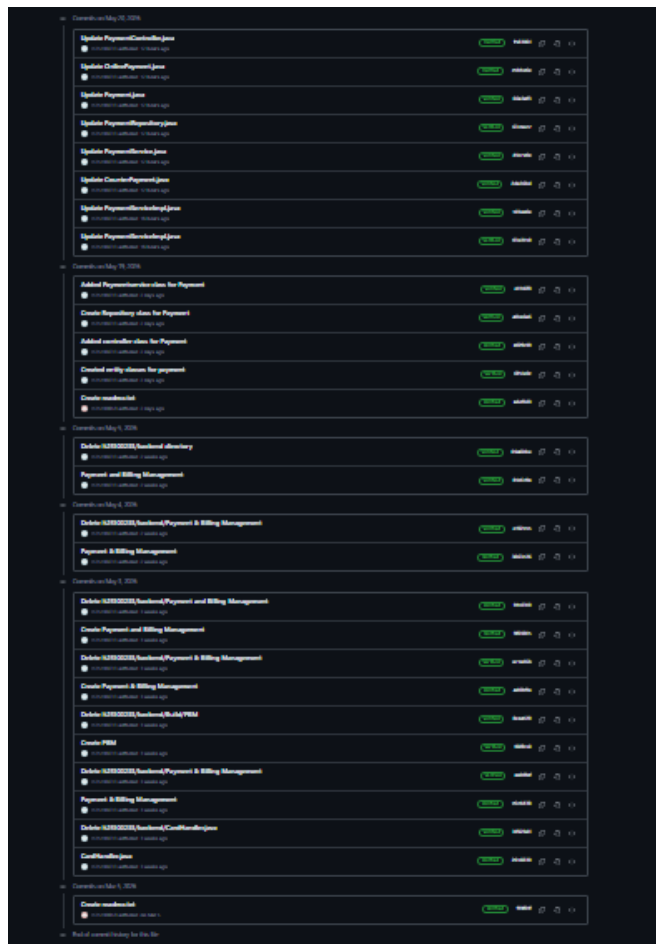
IT25101491 – H.M.K.M Herath



This screenshot displays a GitHub commit history for the user 'H.M.K.M Herath'. The commits are organized by date, with sections for May 20, 2024, May 19, 2024, and May 18, 2024. The commits are primarily related to the 'Reservation & Seat Booking' component, with titles such as 'Reservation & Seat Booking - Final Submission of Component 04', 'Created README documentation for component 04', and 'Added UI screenshots for component 04'. Each commit entry includes a commit hash, a 'Verified' status, and icons for viewing the commit details, the diff, and the files changed.

Commit Date	Commit Title	Commit Hash	Status
May 20, 2024	Reservation & Seat Booking - Final Submission of Component 04	a4c5384	Verified
May 19, 2024	Created README documentation for component 04	3d5b44c	Verified
May 19, 2024	Added UI screenshots for component 04	8db5522	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	a471d38	Verified
May 18, 2024	Reservation & Seat Booking - Component 04	42a4b54	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	886a5d7	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	7d58b3c	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	231c583	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	a704dc6	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	0a6752c	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	c485d47	Verified
May 18, 2024	Reservation & Seat Booking - Final Submission of Component 04	3d5b44c	Verified
May 18, 2024	Kaizara Maleesha - Final Submission of Component 04	c485d47	Verified

IT25100233 – S.M.K.D Senanayake



This screenshot displays a GitHub commit history for the user 'S.M.K.D Senanayake'. The commits are organized by date, with sections for May 20, 2024, May 19, 2024, May 5, 2024, May 4, 2024, and May 3, 2024. The commits are primarily related to the 'Payment & Billing Management' component, with titles such as 'Update PaymentCardDetails.java', 'Update CardInfoPayment.java', 'Add PaymentCardDetails for Payment', 'Create Repository class for Payment', 'Add CardDetails class for Payment', 'Create CardInfo class for payment', 'Create CardInfo.java', 'Delete N25100233,Backend,library', 'Payment and Billing Management', 'Delete N25100233,Backend(Payment & Billing Management)', 'Payment & Billing Management', 'Delete N25100233,Backend(Payment and Billing Management)', 'Create Payment and Billing Management', 'Delete N25100233,Backend(Payment & Billing Management)', 'Create Payment & Billing Management', 'Delete N25100233,Backend(Rule/PBA)', 'Create PBA', 'Delete N25100233,Backend(Payment & Billing Management)', 'Payment & Billing Management', 'Delete N25100233,Backend(CreatePayment.java)', 'CreatePayment.java', and 'Create CardInfo.java'. Each commit entry includes a commit hash, a 'Verified' status, and icons for viewing the commit details, the diff, and the files changed.

Commit Date	Commit Title	Commit Hash	Status
May 20, 2024	Update PaymentCardDetails.java	3d5b44c	Verified
May 20, 2024	Update CardInfoPayment.java	886a5d7	Verified
May 20, 2024	Update Payment.java	a704dc6	Verified
May 20, 2024	Update PaymentPayment.java	0a6752c	Verified
May 20, 2024	Update PaymentDetails.java	231c583	Verified
May 20, 2024	Update CardInfoPayment.java	7d58b3c	Verified
May 20, 2024	Update PaymentDetails.java	c485d47	Verified
May 20, 2024	Update PaymentDetails.java	3d5b44c	Verified
May 19, 2024	Add PaymentCardDetails for Payment	a471d38	Verified
May 19, 2024	Create Repository class for Payment	42a4b54	Verified
May 19, 2024	Add CardDetails class for Payment	8db5522	Verified
May 19, 2024	Create CardInfo class for payment	a4c5384	Verified
May 19, 2024	Create CardInfo.java	3d5b44c	Verified
May 5, 2024	Delete N25100233,Backend,library	a471d38	Verified
May 5, 2024	Payment and Billing Management	42a4b54	Verified
May 4, 2024	Delete N25100233,Backend(Payment & Billing Management)	a471d38	Verified
May 4, 2024	Payment & Billing Management	42a4b54	Verified
May 3, 2024	Delete N25100233,Backend(Payment and Billing Management)	a471d38	Verified
May 3, 2024	Create Payment and Billing Management	42a4b54	Verified
May 3, 2024	Delete N25100233,Backend(Payment & Billing Management)	a471d38	Verified
May 3, 2024	Create Payment & Billing Management	42a4b54	Verified
May 3, 2024	Delete N25100233,Backend(Rule/PBA)	a471d38	Verified
May 3, 2024	Create PBA	42a4b54	Verified
May 3, 2024	Delete N25100233,Backend(Payment & Billing Management)	a471d38	Verified
May 3, 2024	Payment & Billing Management	42a4b54	Verified
May 3, 2024	Delete N25100233,Backend(CreatePayment.java)	a471d38	Verified
May 3, 2024	CreatePayment.java	42a4b54	Verified
May 3, 2024	Create CardInfo.java	a471d38	Verified

IT25100365 – M.R.F Mohamed

Commits on May 20, 2020		
Movie Review and Management	Verified	Health
• IT25100365/backend/MovieReview &ip	Verified	8947568
• IT25100365/backend/MovieReview &ip directory	Verified	8947568
Commits on Apr 25, 2020		
Movie Review and Management	Verified	Health
• IT25100365/backend/last month	Verified	8947568
Add files via upload	Verified	8947568
• IT25100365/backend/last month	Verified	8947568
Commits on Mar 5, 2020		
Create readme.txt	Verified	Health
• IT25100365/backend/on Mar 5	Verified	8947568
End of commit history for this file		

IT25100859 – A.M.T.D Adikari

Commits on May 19, 2020		
Updated ShowTheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Updated ShowTheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Updated ShowTheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Updated ShowTheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Updated ShowTheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Updated ShowTheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Updated ShowTheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on Apr 18, 2020		
Created the ShowTheaterController	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on Apr 18, 2020		
Updated ShowTheaterInventory and TheaterInventory.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on Apr 17, 2020		
Added ShowTheaterInventory Implementation	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on Apr 16, 2020		
Created ShowTheaterInventory Interface	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on May 19, 2020		
Updated the ShowTheaterInventory	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on May 19, 2020		
Created the ShowTheaterInventory class	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on May 18, 2020		
Created the ShowTheater class	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on May 18, 2020		
TheaterInventory class	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Updated the movie ShowTheater.jsp	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Add ShowTheater screen for movie reservation pricing	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
Commits on Mar 5, 2020		
Create readme.txt	Verified	Health
• IT25100859/backend - 1 day ago	Verified	8947568
End of commit history for this file		

IT25102825 – M.R.M Rifky

Commits on May 19, 2020		
Add Controller to find movie data	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Implement Controller for movie file operations	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Add MovieServlet for movie management	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Implement ImageController to serve image files	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Add HomeController for movie gallery functionality	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Add abstract Movie class with properties and methods	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Add GeneralMovie class extending Movie	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Add ComedyMovie class with extends Movie	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Add ActionMovie class extending Movie	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Delete IT25102825/backend/MovieCatalogFinal directory	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Commits on Apr 18, 2020		
Updated MovieCatalogFinal	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Delete IT25102825/backend/MovieCatalog directory	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Commits on Apr 16, 2020		
Add MovieCatalog	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
Commits on Mar 5, 2020		
Create readme.txt	Verified	Health
• IT25102825/backend - 1 day ago	Verified	8947568
End of commit history for this file		

View commit history for this file

updated emailServiceImpl.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated emailService.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated userServiceImpl.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated userService.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
Add file via upload	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated UserController.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated Options.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated User.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated customer.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
updated admin.java	20230503.08	authored 2 days ago	100%	W4554a	🔗
Add file via upload	20230503.08	authored 2 days ago	100%	W4554a	🔗
Add file via upload	20230503.08	authored 2 days ago	100%	W4554a	🔗

Commits on May 4, 2023

Add file via upload	20230503.08	authored 2 weeks ago	100%	W4554a	🔗
Add file via upload	20230503.08	authored 2 weeks ago	100%	W4554a	🔗

Commits on May 5, 2023

Add file via upload	20230503.08	authored 2 weeks ago	100%	W4554a	🔗
Add file via upload	20230503.08	authored 2 weeks ago	100%	W4554a	🔗

Commits on May 1, 2023

Add file via upload	20230503.08	authored 2 weeks ago	100%	W4554a	🔗
Add file via upload	20230503.08	authored 2 weeks ago	100%	W4554a	🔗
Add file via upload	20230503.08	authored 2 weeks ago	100%	W4554a	🔗

Commits on Mar 5, 2023

Create README.txt	20230305.08	authored on Mar 5	100%	W4554a	🔗
-------------------	-------------	-------------------	------	--------	---

End of commit history for this file

